Colegiul Național "B.P. Hasdeu"

Lucrare practică de atestare a cunoștințelor de informatică

Chiticontrol
-
Mini maşină

Profesor îndrumător:
Voicu Amalia-Diana
Clasa a XII-a B

Elevi:
Chitic
David-Alexandru

Data: Mai 2025

Cuprins

I. Prezentarea lucrării 4

II. Tehnologii utilizate 4

II.1 C++ 4

II.2 Java 5

II.3 Android Studio 5

II.4 ESP32 6

III. Schema conceptuală a lucrării 6

IV. Testarea în timp real 8

V. Anexe 10

VI. Bibliografie și Copyright 22

I. Prezentarea lucrarii

În contextul automatizării și controlului dispozitivelor mobile, un rol semnificativ îl joacă proiectarea și implementarea sistemelor de comandă wireless. Atât aplicațiile industriale, cât și cele recreaționale beneficiază de posibilitatea de a opera roboți și vehicule de la distanță, oferind flexibilitate și accesibilitate sporite.

Proiectul de față explorează dezvoltarea unui astfel de sistem, concentrându-se pe crearea **unui** robot mobil cu patru roți, controlat prin intermediul unei aplicații Android. Robotul este echipat cu un sistem de propulsie pe roțile din spate (RWD) și un servomotor pentru direcția roților din față, permițând o manevrabilitate eficientă.

Elementul central al sistemului este placa ESP32, un microcontroler puternic care asigură comunicarea wireless prin Bluetooth. Un aspect distinctiv al acestui proiect îl reprezintă implementarea de la zero a sistemului de comunicare Bluetooth, atât la nivelul aplicației Android (dezvoltată în Java), cât și la nivelul microcontrolerului ESP32 (programat în C++). Pentru a facilita reutilizarea codului și a îmbunătăți structura proiectului, au fost create clase dedicate pentru gestionarea comunicării Bluetooth: **BluetoothManager (Java) și btManager (C++)**. Această abordare a permis o optimizare fină a transferului de date și un control precis asupra interacțiunii dintre cele două componente. Aplicația Android dezvoltată specific pentru acest proiect oferă o interfață intuitivă, facilitând controlul precis al robotului și oferind utilizatorului o experiență interactivă și accesibilă.

Prin urmare, proiectul meu demonstrează integrarea eficientă a tehnologiilor hardware și software moderne, cu o atenție specială acordată dezvoltării personalizate și modulare a sistemului de comunicare Bluetooth, pentru a realiza un sistem de control robust și versatil pentru un robot mobil.

II. Tehonologii utilizate

II.1 C++

C++ este un limbaj de programare de nivel mediu, dezvoltat de Bjarne Stroustrup începând cu 1979 ca o extensie a limbajului C. C++ combină caracteristicile programării orientate pe obiecte cu eficiența și controlul asupra hardware-ului oferite de C. Acest lucru îl face un limbaj puternic și versatil, utilizat într-o varietate de domenii, inclusiv dezvoltarea de sisteme de operare, jocuri, aplicații desktop, sisteme embedded și software de înaltă performanță.

C++ suportă atât programarea procedurală, cât și programarea orientată pe obiecte, permițând dezvoltatorilor să aleagă paradigma care se potrivește cel mai bine nevoilor proiectului. Limbajul

oferă control direct asupra gestionării memoriei, ceea ce poate duce la aplicații mai eficiente, dar necesită o atenție sporită din partea programatorului pentru a evita erorile.

De-a lungul timpului, C++ a fost standardizat de ISO, rezultând în multiple versiuni (C++98, C++03, C++11, C++14, C++17, C++20, C++23) care au adus îmbunătățiri și noi caracteristici limbajului. C++ rămâne un limbaj esențial pentru dezvoltatorii care au nevoie de performanță maximă și control asupra resurselor sistemului.

II.2 Java

Java este un limbaj de programare orientat pe obiecte, conceput inițial de James Gosling la Sun Microsystems (ulterior achiziționată de Oracle Corporation) în 1995. Unul dintre principiile fundamentale ale Java este portabilitatea, ilustrată de sloganul "scrie o dată, rulează oriunde" (WORA - Write Once, Run Anywhere). Această caracteristică este obținută prin compilarea codului sursă Java într-un cod intermediar numit bytecode, care este executat de Mașina Virtuală Java (JVM).

JVM este un software care abstractizează sistemul de operare și hardware-ul subiacent, permițând aplicațiilor Java să ruleze pe orice dispozitiv care are o implementare JVM. Java oferă o gamă largă de funcționalități, de la dezvoltarea aplicațiilor desktop și web la aplicații mobile (Android) și sisteme enterprise.

Limbajul Java este cunoscut pentru sintaxa sa clară, gestionarea automată a memoriei (garbage collection) și bibliotecile bogate. A evoluat semnificativ de-a lungul anilor, adăugând suport pentru programarea concurentă, generică și funcțională. Java continuă să fie un limbaj popular în industrie, fiind folosit pe scară largă în dezvoltarea de aplicații complexe și scalabile.

II.3 Android Studio

Android Studio este un mediu de dezvoltare integrat (IDE) oficial pentru dezvoltarea aplicațiilor Android. Este bazat pe IntelliJ IDEA, un IDE Java puternic, și oferă un set complet de instrumente pentru proiectarea, dezvoltarea, testarea și depanarea aplicațiilor Android. Android Studio este dezvoltat și întreținut de Google.

IDE-ul include un editor de cod inteligent, un emulator pentru a testa aplicațiile pe diferite dispozitive Android virtuale, un profiler pentru a analiza performanța aplicației și un sistem de construire flexibil bazat pe Gradle. Android Studio suportă dezvoltarea în limbajele Java și Kotlin, ambele fiind limbaje oficiale pentru dezvoltarea Android.

Android Studio oferă o serie de caracteristici specifice dezvoltării Android, cum ar fi un editor

de layout vizual pentru proiectarea interfețelor utilizator, suport pentru biblioteci Android specifice și instrumente pentru gestionarea resurselor aplicației (imagini, șiruri de caractere etc.). Prin utilizarea Android Studio, dezvoltatorii pot crea aplicații Android de înaltă calitate, optimizate pentru performanță și compatibilitate cu diverse versiuni ale sistemului de operare.

II.4 ESP32

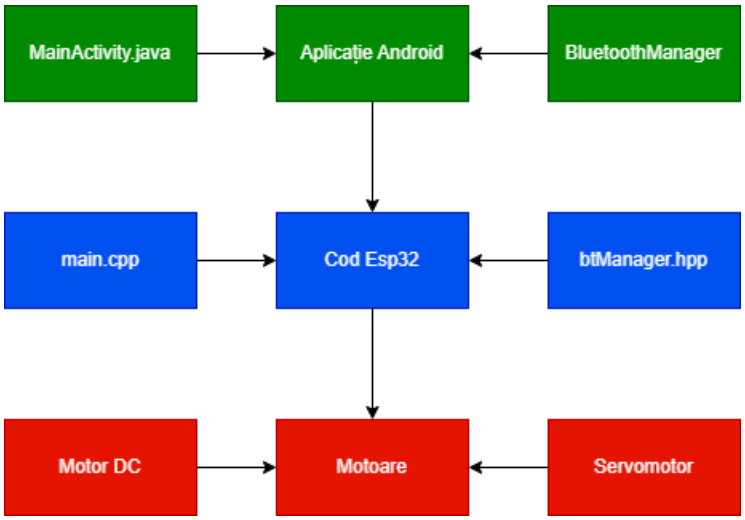
ESP32 este o serie de microcontrolere low-cost, low-power, dezvoltată de Espressif Systems. Aceste microcontrolere integrează conectivitate Wi-Fi și Bluetooth, ceea ce le face ideale pentru aplicații Internet of Things (IoT) și proiecte embedded. ESP32 include un procesor dual-core sau single-core, memorie, periferice (GPIO, UART, SPI, I2C, etc.) și un convertor analog-digital.

ESP32 este programabil în principal folosind limbajele C și C++, cu ajutorul SDK-ului (Software Development Kit) oferit de Espressif. Există, de asemenea, suport pentru alte limbaje și platforme, cum ar fi MicroPython și Arduino. Popularitatea ESP32 se datorează combinației sale de performanță, conectivitate și cost redus, permițând dezvoltatorilor să creeze o gamă largă de dispozitive inteligente și conectate.

ESP32 este utilizat în diverse aplicații, inclusiv automatizarea locuinței, dispozitive purtabile, senzori wireless, control industrial și multe altele. Comunitatea activă de dezvoltatori și resursele disponibile online contribuie la ușurința de utilizare și la extinderea continuă a capacităților acestui microcontroler.

III. Schema conceptuală a lucrării

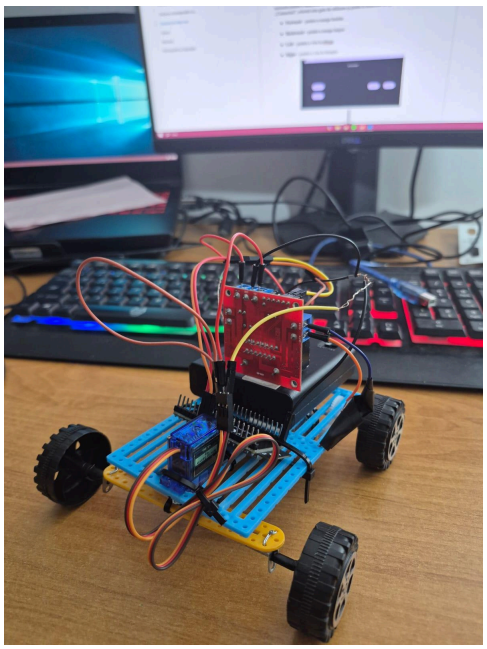
Schema conceptuală a lucrării este după cum urmează

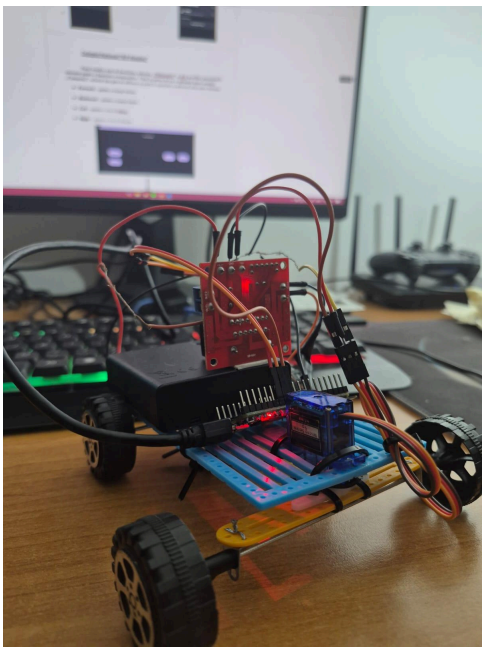


```
.
└── Proiect atestat
    ├── AplicațieAndroid
    │   └── app
    │       └── src
    │           └── main
    │               └── java
    │                   ├── com
    │                   │   └── example
    │                   │       └── myapplication
    │                   │           └── MainActivity.java
    │                   └── custom
    │                       └── BluetoothManager.java
    └── esp32
        ├── include
        │   └── btManager.hpp
        └── src
            └── main.cpp
```

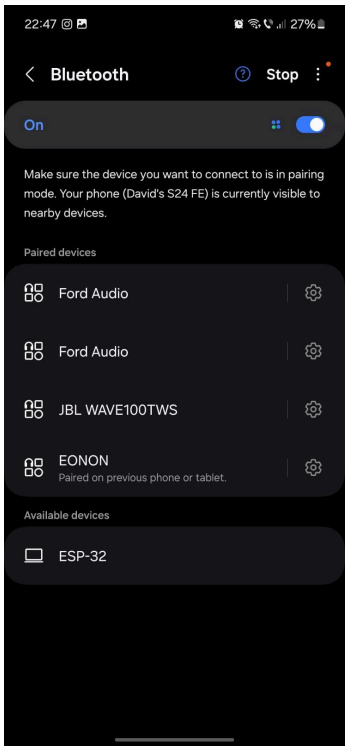
IV. Testarea în timp real

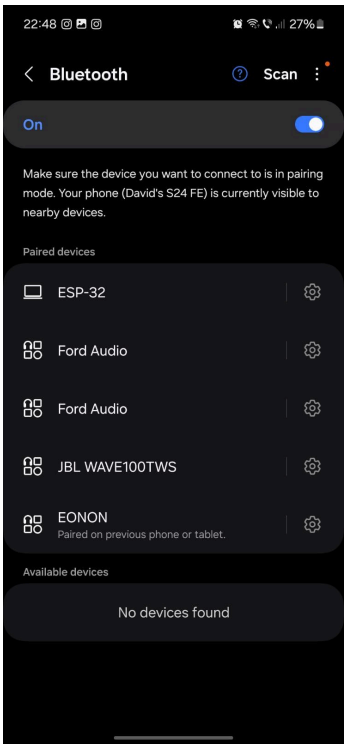
Pentru a utiliza robotul, va trebui să alimentăm placa ESP32 folosind un cablu microUSB și să pornim bateria utilizând întrerupătorul aflat pe aceasta. Două LED-uri roșii se vor aprinde: unul pe placa ESP32 și celălalt pe driverul motorului, semn că ambele componente funcționează corect.





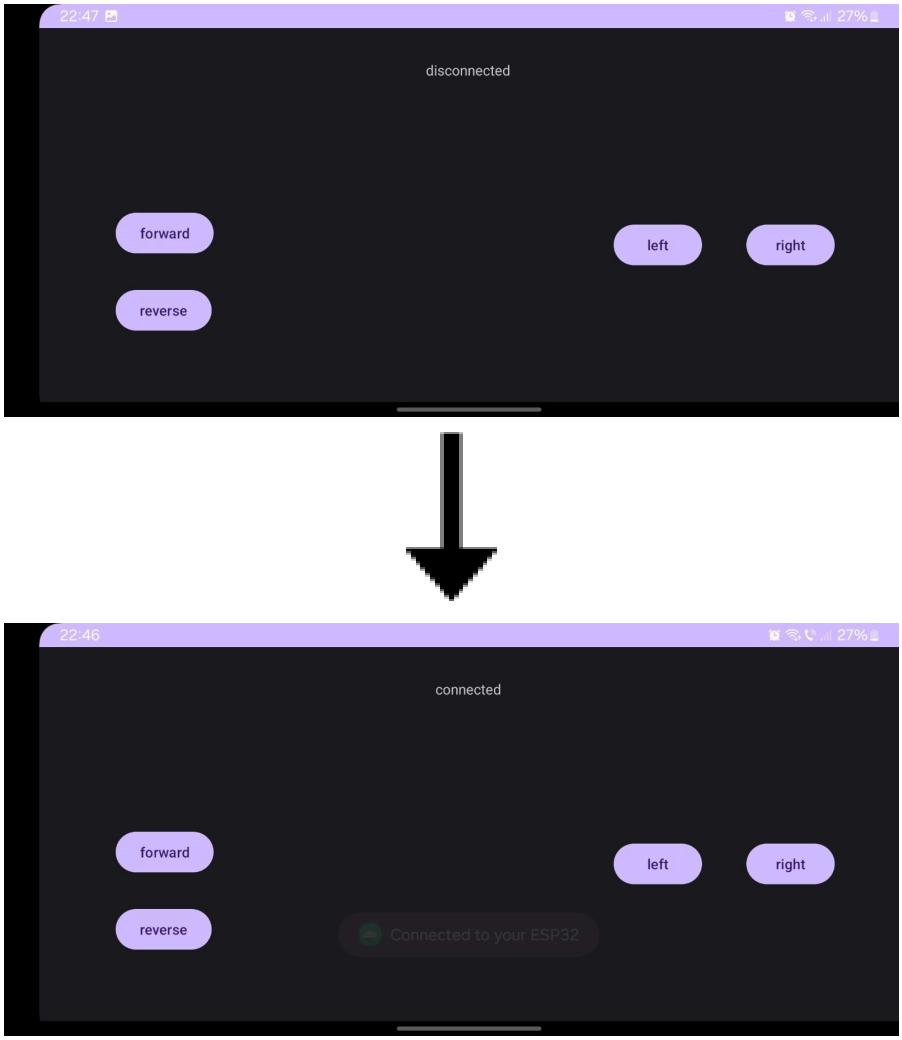
După ce ne asigurăm că partea hardware este operațională, vom activa Bluetooth-ul telefonului și îl vom împerechea cu dispozitivul denumit „ESP-32".





Pasul următor este să deschidem aplicația „Chiticontrol" și să îi acordăm permisiunile necesare pentru a funcționa corespunzător. Când în partea de sus a aplicației apare mesajul „Connected", robotul este gata de utilizare și poate fi controlat cu ajutorul celor patru butoane:

- **Forward** – pentru a merge înainte
- **Backward** – pentru a merge înapoi
- **Left** – pentru a vira la stânga
- **Right** – pentru a vira la dreapta



Videoclip demonstrativ: https://youtu.be/GVRYfFZE36M?feature=shared

V. Anexe

Codul proiectului

- btManager.hpp

```
#include <Arduino.h>
#include <BluetoothSerial.h>
#include <cstring>

BluetoothSerial serialBt;

class btManager{
  public:
    btManager(HardwareSerial& serial) : serial(serial){
      if (!serialBt.begin("ESP-32")) {
        serial.println("An error occurred initializing Bluetooth");
      } else {
        serial.println("Bluetooth initialized");
      }
    }

    void Send(char* message){
      serialBt.println(message);
    }

    void Receieve(char* currentLine) {
      if(serialBt.available()) {
        int character = serialBt.read();

           if (character != '\n' && newLineIndex < 127) { // Check for newline and buffer
overflow
          newLine[newLineIndex++] = (char)character;
          newLine[newLineIndex] = '\0'; // Null-terminate the string
        } else if (character == '\n') {

          strcpy(currentLine, newLine);
          serial.println(currentLine);
          newLineIndex = 0; // Reset index
          newLine[0] = '\0'; // Clear the string
        }
      }
    }

  private:

    HardwareSerial serial;
    char newLine[128] = "";
    int newLineIndex = 0;

};
```

- BluetoothManager.java

```
package custom;

import android.Manifest;
import android.app.Activity;
import android.content.pm.PackageManager;
import android.content.Intent;
import android.bluetooth.*;
import android.widget.TextView;
import android.widget.Toast;

import android.os.HandlerThread;
import android.os.Handler;

import androidx.core.content.ContextCompat;
import androidx.core.app.ActivityCompat;

import java.io.*;
import java.util.UUID;
import android.util.Log;

import com.example.myapplication.R;

public class BluetoothManager {

    HandlerThread ConnectionThread;
    Handler ConnectionHandler;

    HandlerThread ReceiverThread;
    Handler ReceiverHandler;

    private final Activity activity;

    private BluetoothAdapter bluetoothAdapter;
    private BluetoothSocket bluetoothSocket;
    private BluetoothDevice bluetoothDevice;
    private InputStream inputStream;
    private OutputStream outputStream;

                            private       static       final       UUID       MY_UUID       =
UUID.fromString("00001101-0000-1000-8000-00805F9B34FB");
    private static final String esp32_name = "ESP-32";

    private final int PERMISSION_REQUEST_CODE = 1;
    private final int BLUETOOTH_ENABLE_REQUEST = 2;

    private boolean PermissionsGranted = false; // Explicit initialization

    TextView connectionLabel;

    public BluetoothManager(Activity activity) {
        this.activity = activity;
           this.connectionLabel = activity.findViewById(R.id.connectionLabel); // Initialize
here

        ConnectionThread = new HandlerThread("ConnectionThread");
```

```
        ConnectionThread.start();
        ConnectionHandler = new Handler(ConnectionThread.getLooper());

        ReceiverThread = new HandlerThread("ReceiverThread");
        ReceiverThread.start();
        ReceiverHandler = new Handler(ReceiverThread.getLooper());

        checkPermissions();
    }

    public void setPermissionsGranted(boolean value) {
        PermissionsGranted = value;
        if(value) {
            EnableBluetooth();
        }else{
            activity.runOnUiThread(new Runnable() {
                @Override
                public void run() {
                    connectionLabel.setText(R.string.permissionsDeclined);
                }
            });
        }
    }

    public void checkPermissions() {
                                        if     (ContextCompat.checkSelfPermission(activity,
Manifest.permission.BLUETOOTH_CONNECT) != PackageManager.PERMISSION_GRANTED ||
                                                ContextCompat.checkSelfPermission(activity,
Manifest.permission.BLUETOOTH_SCAN) != PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(activity,
                                            new  String[]{Manifest.permission.BLUETOOTH_CONNECT,
Manifest.permission.BLUETOOTH_SCAN},
                    PERMISSION_REQUEST_CODE);
        } else {
            PermissionsGranted = true;
                        Toast.makeText(activity,  "Bluetooth  Permissions  Already  Granted",
Toast.LENGTH_SHORT).show();
            EnableBluetooth(); // Enable bluetooth right away.
        }
    }

    public void EnableBluetooth() {
        bluetoothAdapter = BluetoothAdapter.getDefaultAdapter();

        if (bluetoothAdapter == null) {
            Log.e("Bluetooth", "Bluetooth not supported on this device");
            return;
        }

        // Check if Bluetooth is enabled
        if(!bluetoothAdapter.isEnabled()){
            Intent enableBtIntent = new Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE);
                activity.startActivityForResult(enableBtIntent, BLUETOOTH_ENABLE_REQUEST); //
Request Bluetooth enable
        }

        if(bluetoothAdapter.isEnabled()){
            searchEsp32();
```

```
        }

    }

    private void searchEsp32() {
        ConnectionHandler.post(new Runnable() {
            @Override
            public void run() {
                // First, try to find the ESP32 from paired devices
                bluetoothDevice = getDevice(esp32_name);

                if (bluetoothDevice == null) {
                        // If the device is not found in paired devices, retry after a short
delay
                         Log.d("BluetoothSearch", "ESP32 Device not found in paired devices,
retrying...");
                    ConnectionHandler.postDelayed(this, 100);  // Retry after 3 seconds
                } else {
                    // Bluetooth device found, create connection
                    connectToDevice(bluetoothDevice);
                }
            }
        });
    }

    private void connectToDevice(BluetoothDevice device) {
        ConnectionHandler.post(new Runnable() {
            @Override
            public void run() {
                try {
                    bluetoothSocket = device.createRfcommSocketToServiceRecord(MY_UUID);
                    bluetoothSocket.connect();  // Attempt to connect

                    inputStream = bluetoothSocket.getInputStream();
                    outputStream = bluetoothSocket.getOutputStream();

                    Log.d("BluetoothSearch", "Connected to ESP32");
                    activity.runOnUiThread(new Runnable() {
                        @Override
                        public void run() {
                            connectionLabel.setText(R.string.connected);
                                         Toast.makeText(activity, "Connected to your ESP32",
Toast.LENGTH_SHORT).show();
                        }
                    });

                    startReceiving();

                } catch (IOException e) {
                    Log.e("Bluetooth", "Error connecting or communicating with ESP32", e);
                    activity.runOnUiThread(new Runnable() {
                        @Override
                        public void run() {
                            connectionLabel.setText(R.string.disconnected);
                        }
                    });
                    // Retry if connection failed
                    Log.d("BluetoothConnection", "Retrying connection in 0.1 seconds...");
```

```
                    ConnectionHandler.postDelayed(this, 100);  // Retry after 1 seconds
                }
            }
        });
    }
    // Helper method to get Bluetooth device by name
    private BluetoothDevice getDevice(final String deviceName) {
        for (BluetoothDevice device : bluetoothAdapter.getBondedDevices()) {
            if (device.getName().equals(deviceName)) {
                return device;
            }
        }
        return null;
    }

    //Send
    public void Send(String messageToSend) {
        if(checkConnection()){
            try{
                outputStream.write((messageToSend+"\n").getBytes());
                Log.d("Bluetooth", "Sent: " + messageToSend);
            }catch (IOException e) {
                Log.e("BluetoothSend", "Error sending to ESP32", e);
            }
        }else{
                                Toast.makeText(activity,  esp32_name  +  "  not  connected",
Toast.LENGTH_SHORT).show();
        }

    }

    //Recieve
    public void processReceived(String receivedData) {
        // Update UI if needed (use runOnUiThread)
        activity.runOnUiThread(() -> {
            Toast.makeText(activity, "Received: " + receivedData, Toast.LENGTH_SHORT).show();

        });
    }

    public void startReceiving() {
        if (inputStream == null) {
            Log.e("Bluetooth", "InputStream is null. Cannot start receive thread.");
            return;
        }

        ReceiverHandler.post(new Runnable() {
            @Override
            public void run() {
                try {
                    byte[] buffer = new byte[1024];
                    int bytes;
                    while (checkConnection() && (bytes = inputStream.read(buffer)) != -1) {
                        String receivedData = new String(buffer, 0, bytes);
                        Log.d("BluetoothDebug", "Received: " + receivedData);
                        processReceived(receivedData);
                    }
                } catch (IOException e) {
```

```
                    Log.e("Bluetooth", "Error receiving data", e);
                    searchEsp32(); // Attempt to reconnect if stream is broken
                }
            }
        });
    }

    public boolean checkConnection(){
        return bluetoothSocket != null && bluetoothSocket.isConnected();
    }

    public void Destroy() {
        try {
            if (inputStream != null) {
                inputStream.close();
            }
            if (outputStream != null) {
                outputStream.close();
            }
            if (bluetoothSocket != null) {
                bluetoothSocket.close();
            }
        } catch (IOException e) {
            Log.e("BluetoothManager", "Error closing resources", e);
        }
    }
}
```

- MainActivity.java

```
package com.example.myapplication;

import custom.BluetoothManager;

import android.content.pm.PackageManager;
import android.os.Bundle;
import android.os.Handler;
import android.os.HandlerThread;
import android.util.Log;
import android.widget.*;
import android.view.View;
import android.view.MotionEvent;
import androidx.appcompat.app.AppCompatActivity;
import android.content.Intent;

public class MainActivity extends AppCompatActivity {

    /*HandlerThread backgroundThread;
    Handler backgroundHandler;
    Runnable backgroundRunnable = new Runnable(){
        @Override
        public void run() {
            if(bt.checkConnection()){
                connectionLabel.setText(R.string.connected);
            }else{
                connectionLabel.setText(R.string.disconnected);
            }
```

```
        }
    };*/

    private BluetoothManager bt;
    private Button forward, reverse, left, right;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        /*backgroundThread = new HandlerThread("backgroundThread");
        backgroundThread.start();
        backgroundHandler = new Handler(backgroundThread.getLooper());*/


        bt = new BluetoothManager(this);
        //backgroundHandler.post(backgroundRunnable);
        bt.startReceiving();

        forward = findViewById(R.id.Accelerate);
        reverse = findViewById(R.id.Reverse);
        left = findViewById(R.id.Left);
        right = findViewById(R.id.Right);

        forward.setOnTouchListener(this::Forward_Touch);
        reverse.setOnTouchListener(this::Reverse_Touch);
        left.setOnTouchListener(this::Left_Touch);
        right.setOnTouchListener(this::Right_Touch);
    }

    boolean forwardButtonPressed = false;
    boolean reverseButtonPressed = false;
    boolean leftButtonPressed = false;
    boolean rightButtonPressed = false;

    boolean Forward_Touch(View v, MotionEvent event) {
        switch (event.getAction()) {
            case MotionEvent.ACTION_DOWN:
                forwardButtonPressed = true;
                if (bt.checkConnection()) {
                    bt.Send("forward");
                } else {
                                Toast.makeText(this, "No device connected through bluetooth",
Toast.LENGTH_SHORT).show();
                }
                return true;
            case MotionEvent.ACTION_UP:
                if (forwardButtonPressed) {
                    forwardButtonPressed = false;
                    if (bt.checkConnection()) {
                        bt.Send("stop");
                        Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;//SA FAC ASTA SI LA RESTUL BUTOANELOR*/

            case MotionEvent.ACTION_CANCEL:
```

```
                if (forwardButtonPressed) {
                    forwardButtonPressed = false;
                    Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    if (bt.checkConnection()) {
                        bt.Send("stop");
                        Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;
        }
        return false;
    }

    boolean Reverse_Touch(View v, MotionEvent event) {
        switch (event.getAction()) {
            case MotionEvent.ACTION_DOWN:
                reverseButtonPressed = true;
                if (bt.checkConnection()) {
                    bt.Send("reverse");
                } else {
                                Toast.makeText(this, "No device connected through bluetooth",
Toast.LENGTH_SHORT).show();
                }
                return true;
            case MotionEvent.ACTION_UP:
                if (reverseButtonPressed) {
                    reverseButtonPressed = false;
                    if (bt.checkConnection()) {
                        bt.Send("stop");
                        Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;//SA FAC ASTA SI LA RESTUL BUTOANELOR*/

            case MotionEvent.ACTION_CANCEL:
                if (reverseButtonPressed) {
                    reverseButtonPressed = false;
                    Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    if (bt.checkConnection()) {
                        bt.Send("stop");
                        Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;
        }
        return false;
    }

    boolean Left_Touch(View v, MotionEvent event) {
        switch (event.getAction()) {
            case MotionEvent.ACTION_DOWN:
                leftButtonPressed = true;
                if (bt.checkConnection()) {
                    bt.Send("left");
                } else {
                                Toast.makeText(this, "No device connected through bluetooth",
Toast.LENGTH_SHORT).show();
                }
```

```
                return true;
            case MotionEvent.ACTION_UP:
                if (leftButtonPressed) {
                    leftButtonPressed = false;
                    if (bt.checkConnection()) {
                        bt.Send("stopSteering");
                        //Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;

            case MotionEvent.ACTION_CANCEL:
                if (leftButtonPressed) {
                    leftButtonPressed = false;
                    if (bt.checkConnection()) {
                        bt.Send("stopSteering");
                        Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;
        }
        return false;
    }

    boolean Right_Touch(View v, MotionEvent event) {
        switch (event.getAction()) {
            case MotionEvent.ACTION_DOWN:
                rightButtonPressed = true;
                if (bt.checkConnection()) {
                    bt.Send("right");
                } else {
                                Toast.makeText(this, "No device connected through bluetooth",
Toast.LENGTH_SHORT).show();
                }
                return true;
            case MotionEvent.ACTION_UP:
                if (rightButtonPressed) {
                    rightButtonPressed = false;
                    if (bt.checkConnection()) {
                        bt.Send("stopSteering");
                        Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;

            case MotionEvent.ACTION_CANCEL:
                if (rightButtonPressed) {
                    rightButtonPressed = false;
                    if (bt.checkConnection()) {
                        bt.Send("stopSteering");
                        Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;
        }
        return false;
    }
```

```
    @Override
      public void onRequestPermissionsResult(int requestCode, String[] permissions, int[]
grantResults) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults);

        if (requestCode == 1) {
            boolean allGranted = true;
            for (int result : grantResults) {
                if (result != PackageManager.PERMISSION_GRANTED) {
                    allGranted = false;
                    break;
                }
            }
            if (allGranted) {
                Log.d("acces", "Permissions granted");
                Toast.makeText(this, "Permissions Granted", Toast.LENGTH_SHORT).show();
                  bt.setPermissionsGranted(true);  // Proceed with Bluetooth connection after
permissions are granted
            } else{
                bt.setPermissionsGranted(false);
                Toast.makeText(this, "Permissions Denied", Toast.LENGTH_SHORT).show();
            }
        }
    }

    // Handle Bluetooth enabling result
    @Override
    protected void onActivityResult(int requestCode, int resultCode, Intent data) {
        super.onActivityResult(requestCode, resultCode, data);
        if (requestCode == 2) { // Bluetooth enable request
            if (resultCode == RESULT_OK) {
                 // bt.connectToEsp32();  // Proceed with Bluetooth connection after enabling
Bluetooth
            } else {
                Toast.makeText(this, "Bluetooth not enabled", Toast.LENGTH_SHORT).show();
            }
        }
    }
}
```

- main.cpp

```
#include <btManager.hpp>
#include <ESP32Servo.h>

Servo servo;
int forwardMotor = 12, reverseMotor = 14;
int m = 0, s = 0, steeringAngle;
bool isSteering;
btManager *bt = nullptr;
char *message = new char[128];

void processMessage(const char* message);
void drive();

void setup() {
  Serial.begin(9600);
```

```
  bt = new btManager(Serial);
  pinMode(forwardMotor, OUTPUT);
  pinMode(reverseMotor, OUTPUT);
  digitalWrite(forwardMotor, LOW);
  digitalWrite(reverseMotor, LOW);
  servo.attach(27);
  steeringAngle = 100;
  servo.write(steeringAngle);
  isSteering = false;
}

void loop() {
  bt->Receieve(message);
  processMessage(message);
  drive();
}

void processMessage(const char* message){
      if(strcmp(message, "forward") == 0) m=1;
      else if(strcmp(message, "reverse") == 0) m=-1;
      else if(strcmp(message, "stop") == 0) m=0;


      if(strcmp(message, "left") == 0){
        s=-1;
        isSteering = true;
      }
      else if(strcmp(message, "right") == 0){
        s=1;
        isSteering = true;
      }
      else if(strcmp(message, "stopSteering") == 0){
        s=0;
        isSteering = true;
      }

    }

void drive(){
  switch(m){
    case 1:
      digitalWrite(forwardMotor, HIGH);
      digitalWrite(reverseMotor, LOW);
      break;
    case -1:
      digitalWrite(forwardMotor, LOW);
      digitalWrite(reverseMotor, HIGH);
      break;
    default:
      digitalWrite(forwardMotor, LOW);
      digitalWrite(reverseMotor, LOW);
      break;
  }

  if(isSteering){
    switch(s){
    case 1:
      steeringAngle = 120;
```

```
      break;
    case -1:
      steeringAngle = 80;
      break;
    default:
      steeringAngle = 100;
      break;
    }
    servo.write(steeringAngle);
    isSteering = false;
  }

}
```

VI. Bibliografie și Copyright

- https://en.wikipedia.org/wiki/Java_(programming_language)
- https://en.wikipedia.org/wiki/C%2B%2B
- https://en.cppreference.com/w/
- https://en.wikipedia.org/wiki/Android_Studio
- https://developer.android.com/studio
- https://randomnerdtutorials.com/